

UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Corso di Laurea in Informatica
Programmazione ad Oggetti

Relazione di progetto

GESTORE DI ATTIVITÀ
AGENDA++

Candidato: Francesco Mazzurana, francesco.mazzurana@studenti.unipd.it

Matricola: 2149463

Docenti:

Prof. Francesco Ranzato

Prof. Marco Zanella

Anno Accademico 2025-2026

Indice

1 Introduzione 1

2 Descrizione del modello 1

3 Polimorfismo 4

4 Persistenza dei dati 5

5 Funzionalità implementate 6

6 Rendicontazione ore 7

1 Introduzione

Agenda++ è un'applicazione desktop per la gestione di attività di vario tipo, pensata in maniera specifica per le esigenze di uno studente universitario. Il programma non permette solo la classica gestione CRUD (Create, Read, Update, Delete) di attività, ma introduce anche funzionalità specifiche quali la gestione degli esami universitari con relativo libretto con funzionalità utili, appuntamenti, scadenze a step (con delle task da completare) e attività sportive ricorrenti, ciascuna con proprietà specifiche e visualizzazione dedicata. Inoltre ho posto particolare enfasi nella ricerca/filtraggio, che è real-time e offre una vasta gamma di filtri che permettono di combinare in AND vari criteri (titolo, tipo, coordinate temporali, urgenza, tag ecc.) in maniera flessibile, restituendo solo risultati pertinenti, semplicemente popolando i campi di ricerca senza dover confermare o fare azioni particolari.

Altri punti nei quali ho riposto parecchio tempo e attenzione in fase di sviluppo sono stati i principi di polimorfismo, e l'implementazione di vari pattern, sia per impararli, che per rendere l'architettura più solida ed estendibile nel tempo, inoltre, come richiesto da specifiche, ho separato il modello logico dall'interfaccia grafica, con l'unica eccezione concessa di usare tipi e metodi forniti da QT per gestione salvataggio in JSON e per i tipi utili al modello logico, slegati dalla parte grafica.

Questo progetto prende ispirazione, (con le giuste semplificazioni), dalle app calendario e promemoria di Apple, l'app myUniPd, e alcuni elementi di piattaforme di e-commerce quali ebay per la ricerca, facendone un mix delle parti utili.

Ho scelto questo progetto perchè essendo la mia prima esperienza di programmazione a questo livello ho voluto cercare di variegare le componenti implementate, cercando di imparare quanto più possibile, adempiendo in questo modo anche alle richieste di uso di polimorfismo in maniera non banale delle specifiche.

2 Descrizione del modello

Il cuore del modello logico ruota attorno alla gerarchia di **Attivita.** La classe base astratta **Attivita** raggruppa le caratteristiche comuni (titolo, id, note, data e dataCreazione, durata, priorita, vettore dei tag e stato di completamento) con tutti i relativi getter e setter, per ottenere tutti gli elementi utili ed eventualmente modificarli, con la possibilità di aggiungere tag, inoltre vi è il metodo accept, necessario per implementare il design pattern Visitor attraverso le due interfacce, const e non const , e due metodi polimorfi isUrgente, virtuale puro, che restituisce un booleano per indicare lo stato di urgenza in base a vari parametri con comportamento specifico in base al tipo, e avvieneInData, virtuale, che riceve in ingresso un QDate, e restituisce un bool che indica se l'attivita avviene o meno nel giorno passato, implementato direttamente in **Attivita**, e con l'unico override in Sport, che essendo ricorsiva nel tempo assume un comportamento diverso. (Questo metodo è aperto ad eventuali estensioni del modello con nuove tipologie di attività che possono avere ripetizioni e eventi temporali diversi). È stato inoltre implementato il clone pattern, anche se dopo non è stato effettivamente utilizzato, quindi lo considero secondario.

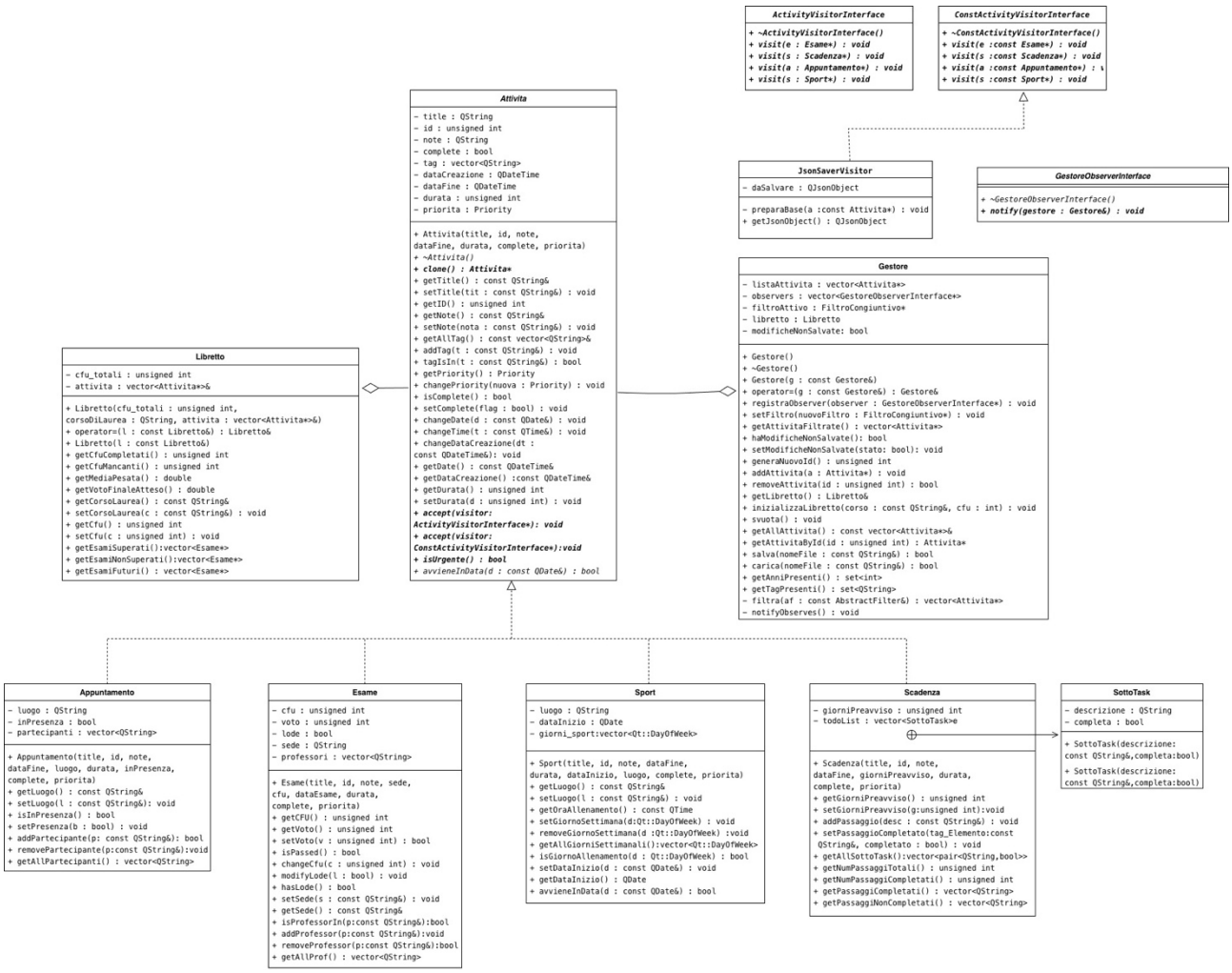


Figura 1: UML del modello logico

Da **Attivita** derivano quattro classi concrete:

- **Esame**: estende **Attivita** rappresentando un esame universitario mediante gli attributi CFU, voto conseguito, con possibilità di lode, sede, elenco dei professori del corso, e stato di superamento dell'esame, anche qua con relativi getter e setter e metodi per interfacciarsi col vettore presente, visibili in figura 1, i nomi dei metodi sono autodescrittivi. L'implementazione di isUrgente() di Esame si basa sull'avvicinarsi della data, pesando l'urgenza in base alla priorità impostata e al carico di studio richiesto(CFU).

- **Appuntamento**: gestisce un incontro definendo gli attributi per rappresentare il luogo, se è in presenza o on-line, i partecipanti mediante la loro e-mail (validata in fase di inserimento da una espressione regolare), anche qua i metodi presenti sono i vari getter e setter, più dei semplici metodi per interfacciarsi col vettore dei partecipanti ecc. L'implementazione di isUrgente() in questa classe si basa invece che sui giorni mancanti sulle ore mancanti, in particolare più alta sarà la priorità prima verrà indicata come urgente.

- **Scadenza**: pensata per i progetti a lungo termine, estende **Attivita** con attributi per gestire i giorni di preavviso, e aggiunge un vettore di **SottoTask** che sono una inner class implementata per rappresentare un task da compiere, ovvero una stringa che rappresenta la cosa da fare e un booleano per indicarne completamento o meno. I metodi come prima sono i vari getter e setter, con l'aggiunta di metodi per cambiare lo stato di completamento dei task e ottenere il numero di task completate e non. L'implementazione di isUrgente() in questa classe avviene mediante la seguente logica: se i giorni mancanti sono meno della soglia dei giorni di preavviso ritorna vero,

cosi come se la priorità è alta, negli altri casi viene indicata come urgente in relazione a priorità e numero di task ancora da completare.

- **Sport**: rappresenta un'attività sportiva ricorrente, aggiungendo per farlo gli attributi dataInizio per indicare quando inizia effettivamente l'attività sportiva, un vettore di Qt::dayOfWeek per indicare i giorni della settimana ove si ripete l'attività e luogo per indicare appunto dove si svolge. Oltre ai getter e setter soliti, vi è un metodo per ottenere l'ora di allenamento, che sarà rappresentata dalla parte time del QDateTime data della base, e i metodi per gestire i vari giorni della settimana, quindi aggiungerli, toglierli e ottenere quelli selezionati. Questa classe fa un override del metodo AvvieneInData della base, qua ritorna vero se la data passata corrisponde a uno dei giorni della settimana selezionati nell'intervallo tra data di inizio e data di fine. L'implementazione di isUrgente() in questa classe avviene con la seguente logica: verifica se il giorno attuale è un giorno di allenamento, ed in base a priorità ed ore mancanti assegna o meno l'urgenza.

(Per tutte le derivate, il metodo isUrgente restituisce falso se l'attività è' completata, e vero se la data è già passata ma l'attività non è stata impostata come completata, per tutti i metodi che si interfacciano con vettori interni, vi è controllo per evitare ripetizioni durante l'inserimento)

Ad orchestrare l'intera logica del modello vi è la classe **Gestore**, essa funge da hub centrale per la mia architettura, e opera da soggetto per il pattern observer (le notify per indicare le variazioni sono emesse qua). Il gestore è l'unico proprietario del ciclo di vita dei dati: detiene il vettore di **Attivita** attualmente presenti ed è responsabile di allocazione e deallocazione della memoria per evitare memory leak, è implementata la rule of three, e si occupa di tutte le logiche del programma: gestire, inserire, rimuovere attività, e tutti i relativi metodi di utilità per gestirli in maniera solida, visibili nel diagramma in figura 1. Gestisce inoltre il caricamento e salvataggio dei dati in formato JSON, per il salvataggio delega la logica al **JsonSaverVisitor**, mentre per il caricamento la gestisce direttamente, approfondito nella parte di persistenza dei dati, ha anche una variabile booleana per indicare la modifica dei dati con relativi metodi di lettura, utilizzata per informare se vi sono modifiche non salvate prima di chiudere il programma o caricare un altro file. Il gestore si occupa inoltre del filtraggio delle attività, delegandone il funzionamento al sistema di interfacce **AbstractFilter**, inoltre gestisce internamente per composizione un oggetto **Libretto**.

La classe **Libretto** rappresenta un modulo di calcolo dominio-specifico: viene interrogata per processare statisticamente le sole attività di tipo Esame contrassegnate come completate. Si occupa in completa autonomia di calcolare metriche fondamentali per lo studente, quali la Media Pesata (tenendo conto della lode), il conteggio dei CFU acquisiti rispetto al totale del Corso di Laurea e la proiezione matematica del Voto di Laurea in centodecimi. Per fare ciò ha un riferimento al vettore delle attività del gestore, inoltre ha un campo per sapere i cfu del corso di laurea e una stringa per salvare il nome del corso, dispone inoltre di metodi per sapere esami superati, non superati e futuri. Questa classe è l'unica che utilizza in alcuni suoi metodi la RTTI, avrei potuto ovviarla con l'uso di un visitor come da altre parti, ma essendo che sfrutta solo il tipo esame, sarebbe stata una implementazione che lasciava vuoti tre visit su 4, perdendo di senso.

Per quanto riguarda il filtraggio, ho usato una architettura basata sulla classe base astratta **AbstractFilter**, la quale oltre al distruttore virtuale ha un unico metodo virtuale puro bool filtra(const Attivita*) che viene implementato nelle varie derivazioni per implementare un filtro diverso, usando il pattern strategy, (descritto successivamente), i filtri presenti, visibili anche nel diagramma in figura 2, sono:

- **FiltroCongiuntivo**: implementa il pattern composite, è un filtro che contiene un vettore di abstractFilter e lavora in AND, restituisce le attività che soddisfano tutti i filtri presenti, ed è quello presente dentro al Gestore, di fatto tutta la logica di filtraggio è basata su un unico grande filtro congiuntivo popolato di volta in volta.

- **FiltroTipo**: implementa il filtraggio per tipo, lo ritengo particolarmente interessante: sfrutta una inner-class **checkVisitor** (derivata da **ConstActivityVisitor**) che, in combinazione a un valore enumerativo, permette di filtrare le attività per tipo senza alcun uso di RTTI (nessun dynamic_cast), rendendo il codice solido e facilmente estendibile.

- **FiltroNome**: implementa il filtraggio per nome, basato sulla ricerca come sottostringa, case-insensitive.

Il resto dei filtri sono: **FiltroData, FiltroAnno, filtroMeseAnno, FIltroDataCreazione, FiltroCompletate, FiltroUrgente, FIltroTag, FiltroPriorita**, visibili sempre in figura 2, i nomi stessi sono descrittivi del funzionamento di ciascuno.

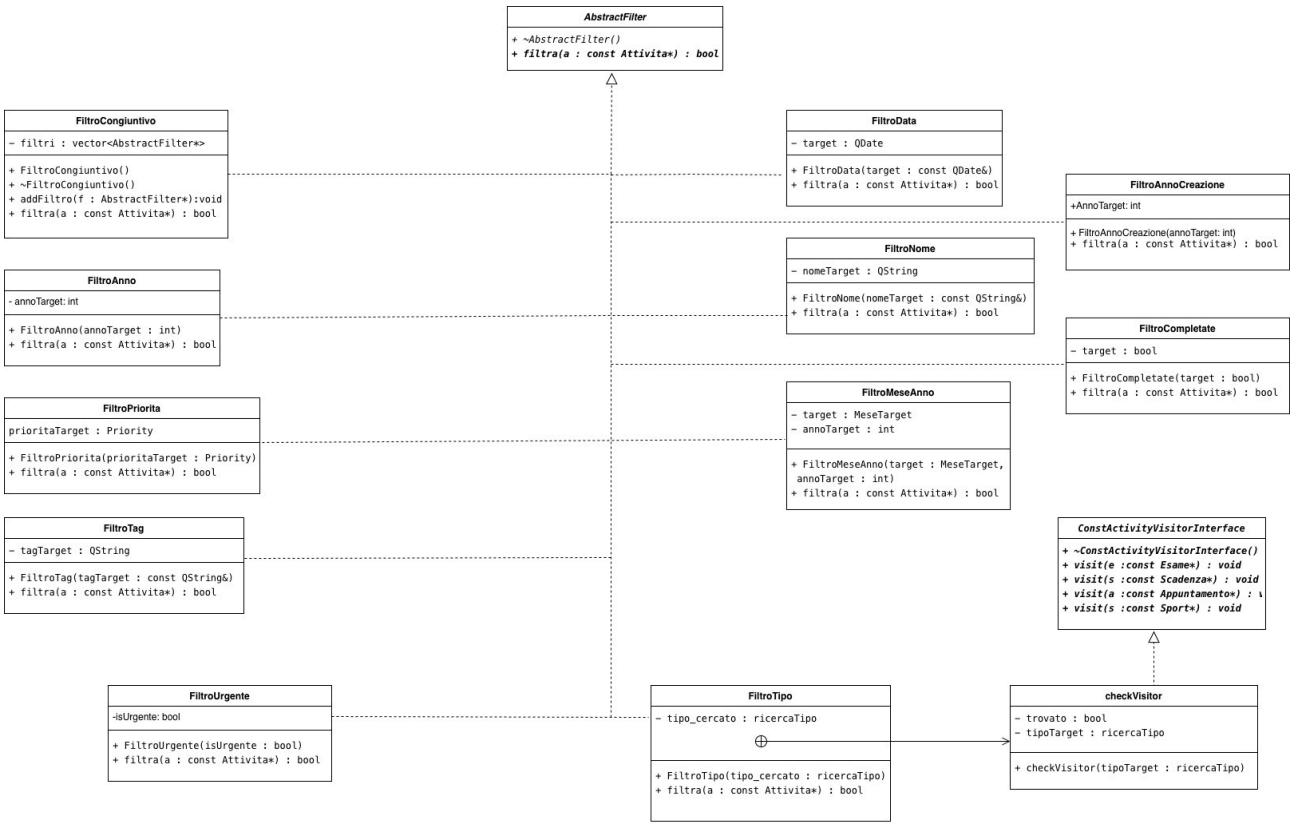


Figura 2: UML dei filtri

3 Polimorfismo

L'utilizzo principale del polimorfismo riguarda il design pattern Visitor nella gerarchia di **Attivita**, che è stato utilizzato in varie parti del programma, innanzitutto sono state create le due interfacce, **ActivityVisitorInterface** e **ConstActivityVisitorInterface.**

I Visitor vengono impiegati per la conversione in JSON: il **JsonSaverVisitor** estrae i campi specifici delle derivate delegando la chiamata di salvataggio al visitatore stesso. Inoltre, come descritto in precedenza, il Visitor è sfruttato internamente da **FiltroTipo** (tramite la inner-class **checkVisitor**) per determinare il tipo di appartenenza senza ricorrere alla RTTI, rendendo il modello robusto ad aggiunte future. Per quanto riguarda la GUI, il polimorfismo permette di istanziare dinamicamente i widget visivi corretti. Il **CardActivityVisitor**, derivato da **ActivityVisitorInterface**, genera le card visibili permettendo azioni interattive dirette (come il cambio di stato di completamento), senza dover entrare nella modalità di modifica. Infine, il **CardEditActivityVisitor** (derivato invece dall'interfaccia const) genera i widget per la schermata di creazione/modifica. Avendo optato per la cancellazione e ri-creazione dell'attività durante le modifiche, l'uso dell'interfaccia const garantisce sicurezza in combinazione con il pattern Factory Method, che descriverò successivamente.

Gli altri design pattern utilizzati sono:

- Il design pattern Strategy, utilizzato nella gerarchia di **AbstractFilter,** di fatto la logica di filtraggio cambia dinamicamente a runtime delegando il check al metodo virtuale puro filtra(const Attivita*), della specifica classe di filtro,(data, tipo, nome, ecc.), istanziata ed assegnata al **Gestore** in un dato momento.

- Il design pattern Composite, che estende la gerarchia del pattern Strategy: la classe **FiltroCongiuntivo** eredita da **AbstractFilter,** ma possiede al suo interno un vettore di ulteriori AbstractFilter*, esso itera e delega in maniera polimorfa le esecuzioni vere e proprie ai vari filtri nel suo vettore, permettendo l'applicazione combinata di n filtri separati, dove n è la size del vettore, in maniera interscambiabile come fosse un filtro singolo. (es "Priorità Alta" AND "Solo Esami")

- Il design pattern Observer, utilizzato per disaccoppiare logicamente il modello logico dalle viste della GUI. Il **Gestore** itera su un vettore di puntatori all'interfaccia astratta **GestoreObserverInterface,** ad ogni azione che comporta delle modifiche agli elementi nel gestore, viene mandata una notify, questa permette alle 3 viste principali, (elenco, calendario, libretto), che derivano da **GestoreObserverInterface,** di venire "ri-disegnate" ogni qualvolta vi sia una modifica che richiede di generare una visualizzazione grafica aggiornata.

- Il design pattern Factory Method, l'interfaccia astratta dei widget di editing/creazione della GUI, **AbstractCardEdit**, definisce il metodo virtuale puro generaAttivita(), delegando alle sue implementazioni derivate,(**CardEsame, CardAppuntamento, CardSport, CardScadenza**) l'istanziazione polimorfa e il popolamento degli oggetti concreti con i dati dell'utente.

4 Persistenza dei dati

Per la persistenza dei dati viene utilizzato il formato JSON, con un unico file per ciascuna agenda di attività.

Il file di storage JSON contiene un JsonObject che contiene i campi specifici del libretto, (nome corso e cfu) e un JsonArray che conterrà il vettore di attività, ove ognuna sarà un oggetto che è un'associazione chiave-valore, con l'eccezione delle classi che contengono vettori interni, in quei casi il valore associato alla chiave è un ulteriore JsonArray di oggetti codificati nel medesimo modo, la serializzazione delle sottoclassi viene gestita aggiungendo un attributo implicito testuale "Tipo". In fase di deserializzazione la classe **Gestore** legge questo campo per capire quale costruttore concreto invocare per ripristinare la gerarchia in modo corretto. Come descritto in precedenza la fase di salvataggio è gestita tramite un visitor, ma sempre sotto competenza del gestore, il caricamento ovviamente avviene senza visitor. Sempre nel gestore vi è un attributo booleano utilizzato per vedere se le attività attualmente presenti sono state salvate, in modo da avvisare l'utente se chiude l'applicazione o prova a caricare un file diverso prima di salvare le modifiche. Un file d'esempio, rappresentativo della struttura dei file è fornito insieme al codice, in particolare "test_agenda.json" contiene tutte le tipologie di attività, in modo da illustrare brevemente le diverse strutture.

La figura 3 mostra un esempio di un pezzo del file JSON fornito insieme al progetto.

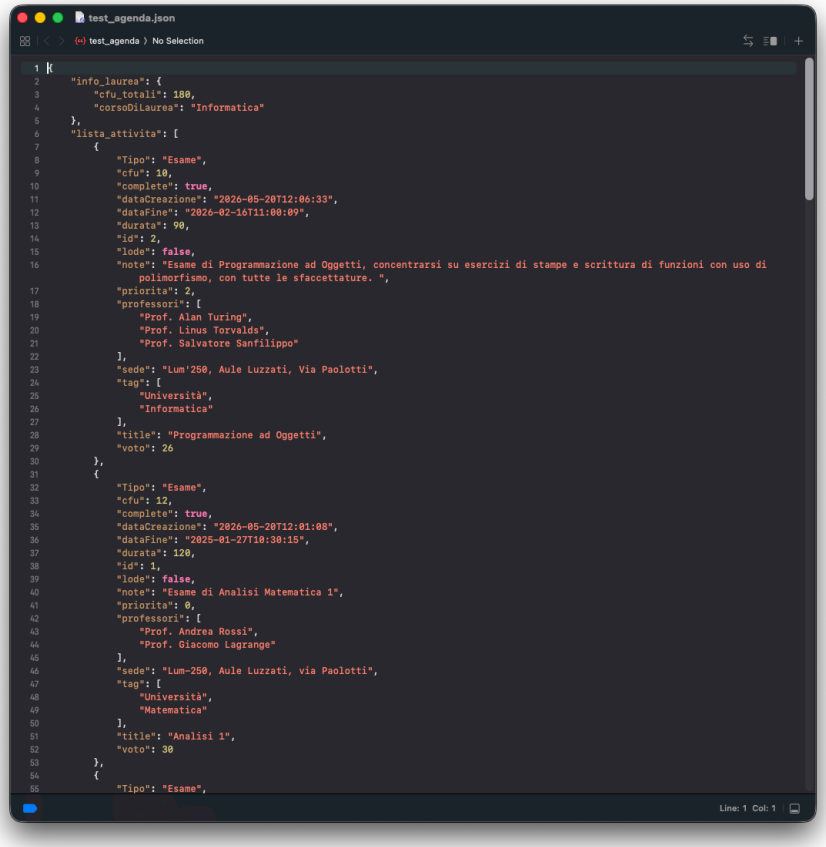


Figura 3: Frammento del file di storage

5 Funzionalità implementate

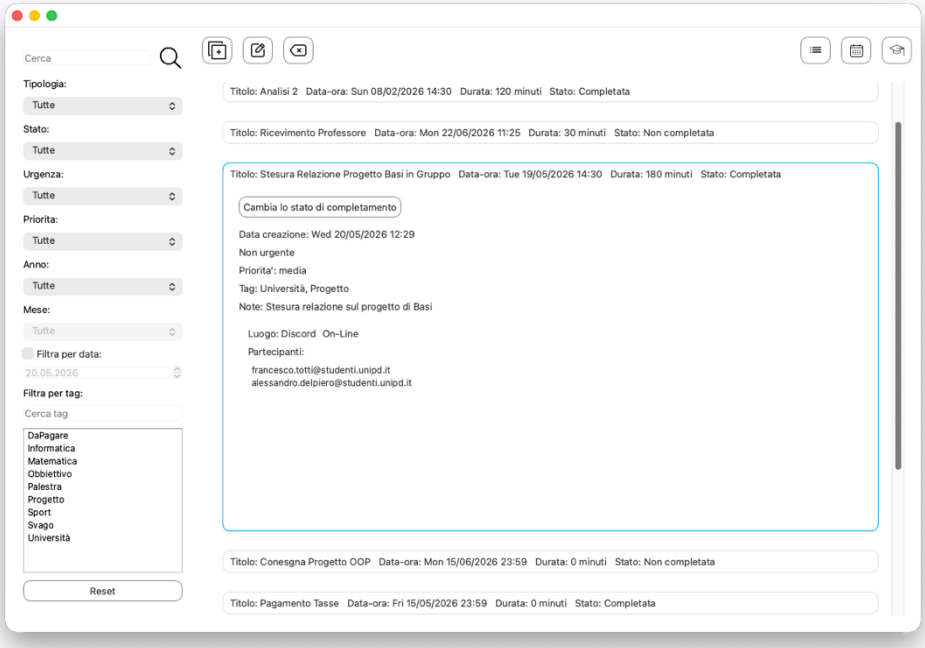


Figura 4: Schermata principale aperta su vista ad elenco con attività espansa

(**nota:** su macOS la barra dei menù non si trova nella finestra dell'applicazione)

Le funzionalità principali, oltre ai requisiti minimi di progetto, si dividono in due categorie:

Funzionali:

- Ricerca avanzata incrociata (mediante **FiltroCongiuntivo**) su 8 attributi differenti contemporaneamente
- Conversione e salvataggio in formato JSON
- Gestione automatizzata del libretto universitario con calcolo in tempo reale di media pesata, CFU acquisiti e proiezione del voto di laurea
- VistaCalendario, che posiziona dinamicamente le attività nei giorni di competenza

Grafiche e di interazione:

- Navigazione single-page tramite l'uso di QStackedWidget per passare da VistaElenco a Calendario, dettaglio o modifica rimanendo nella stessa finestra mediante la pressione di bottoni o attività, per cambiare la modalità di visualizzazione
- Disposizione responsiva del layout e dei pannelli laterali (BarraLateraleWidget) a seconda della vista attuale
- barra dei menù in alto per salvataggio/caricamento dei file e per avere indicazioni sulle scorciatoie da tastiera
- utilizzo di icone nei bottoni
- utilizzo di una toolbar con i bottoni per le viste sempre disponibili e bottoni per aggiunta modifica ed eliminazione di attività variabili in base alla vista

- scorciatoie da tastiera, documentate nell'apposito menù
- Protezione chiusura accidentale attraverso l'override del closeEvent, se ci sono modifiche non salvate l'applicazione intercetta il tentativo di chiusura e avverte l'utente permettendo il salvataggio immediato
- Prevenzione perdita di dati al caricamento, quando si tenta di caricare un nuovo file JSON, il sistema verifica la presenza di modifiche non salvate e richiede conferma, con eventuale salvataggio, prima di procedere, evitando la sovrascrittura dei file attuali
- Controllo coerenza dati inseriti in fase di modifica creazione, con finestre di errore in caso di selezioni irragionevoli (es. impostare come completata un'attività futura ecc.)
- utilizzo di colori e stili grafici di base
- Attività visualizzate in modalità compatta con informazioni di base, espanse con tutte le informazioni al click
- effetti grafici come cambio del colore al passaggio del mouse
- Finestre di conferma per operazioni critiche (Eliminazione, errori di salvataggio)
- Disabilitazione logica di pulsanti modifica/elimina in assenza di selezione attiva di una attività

6 Rendicontazione ore

Attività	Ore Previste	Ore Effettive
Studio e progettazione	8	10*
Sviluppo del codice del modello	10	13*
Studio del framework Qt	8	12
Sviluppo del codice della GUI	16	21
Test e debug	4	8
Stesura della relazione	4	4
totale	50	**68**

La previsione era già ben superiore alle 40 ore richieste idealmente perchè partivo già conscio del fatto che questo era il mio primo progetto serio di programmazione e sapevo sarebbe stato più lungo del tempo auspicabile. Le ore di studio e progettazione e sviluppo del codice del modello in realtà avrebbero rispettato le ore, dato che mi ero lasciato margine, come poi è successo, per modifiche in corso d'opera per idee che mi venivano andando avanti o per funzionalità necessarie alle quali non avevo pensato subito, ad essi si è aggiunta una parte di studio e tempo superiore dato che mi sono imposto di provare ad utilizzare metodi di algorithm o propri dei vettori ecc., insieme ad ampio uso di lambda espressioni, per parti che effettivamente potevano essere risolte in maniera più semplice o più classica, inoltre nel modello vi sono parti totalmente "giocattolo" come il parser con espressione regolare per le mail, dato che è una funzionalità che effettivamente non ha una utilizzo reale nella versione attuale del progetto, o altre parti del modello che avevo pensato essere utili, ma poi in fase di realizzazione nella GUI si sono rivelate troppo complesse e dispendiose in termini di tempo.

Per quanto riguarda lo studio di Qt e la realizzazione della GUI, avevo previsto di impiegarvi molto tempo, dato che non avevo mai realizzato interfacce grafiche, e così effettivamente è stato, ho perso parecchio tempo per predisporre parti che poi ho dovuto semplificare in corso d'opera, come per esempio la vistaCalendario, che originariamente volevo fare copiando la visualizzazione delle lezioni nell'app myUnipd, che mostra le 24 ore dei 7 giorni della settimana visualizzata, e disegna le lezioni in base alle ore occupate, che si è rivelato estremamente complesso, e che avrebbe richiesto anche vincoli più forti nel modello logico, inoltre ho perso molto tempo nell'implementare idee che avevo che

nella realizzazione si sono rivelate dispendiose, come la visualizzazione stessa delle attività con la modalità compatta che al click si espande, che di per se non è complicata, ma che mi ha richiesto tempo per capire di strutturarla con due layout di cui uno che diventa visibile/invisibile al click, e soprattutto capire come fare l'override del mousePressEvent, che da implementare una volta capito ha richiesto poco tempo, ma per capire che dovevo farlo mi ha richiesto ore e svariate prove. La GUI inoltre l'ho cambiata esteticamente più e più volte provandola e variando i layout e le scelte di volta in volta, e questo, unito alla difficoltà iniziale data dall'inesperienza, mi ha portato a perdere svariate ore.

Infine la fase di debug è stata parecchio lunga, prevalentemente per colpa mia, al di la dei classici errori dopo eventuali modifiche, che ho capito essere una costante, ho perso mezza giornata per mie colpe di pigrizia: il pattern observer l'avevo predisposto dall'inizio, però inizialmente ho impostato la GUI con aggiornamenti manuali con l'idea che per poi inserirvi l'observer avrebbe richiesto poche righe e tempo, invece mi sono trovato a dover cambiare l'intera logica di chiamate dei filtri nell'interfaccia grafica, dato che per come l'avevo implementata inizialmente, una volta inserito l'observer compila normalmente, ma dava sempre errori a runtime appena si provava a caricare un file, creare un attività o simile, e questo era dato dalla seguente catena di eventi, che causava stackoverflow, e che per capirla mi ha richiesto uno sforzo esteso:

1. Il Gestore notificava la BarraLaterale
2. La BarraLaterale per aggiornare gli anni chiamava comboAnno->clear()
3. Il clear() svuotava il menu, emettendo il segnale currentIndexChanged
4. Il segnale avvisava la MainWindow che l'utente aveva cambiato un filtro.
5. MainWindow creava un nuovo filtro e chiamava Gestore::setFiltro().
6. Il Gestore notificava di nuovo la BarraLaterale, facendo ripartire il giro all'infinito

Per ovviare ciò ho inserito le chiamate comboAnno->blockSignals(true) prima di modificare l'interfaccia via codice (e blockSignals(false) alla fine). In questo modo Qt capisce che le modifiche non sono azioni dell'utente e non emette segnali, bloccando la ricorsione.

Un problema non causato da me è stato invece che ho programmato questa applicazione su MacOS, in particolare su un dispositivo arm, e questo mi ha dato problemi con alcune parti di Qt, in particolare dopo un aggiornamento dell'OS, dunque anche del compilatore Apple Clang, è iniziato ad uscirmi un errore fisso in fase di compilazione: error: implicitly declaring library function '__yield' with type 'void ()' [-Werror,-Wimplicit-function-declaration]

```
37 |     __yield();              // Generic
   |     ^
```

ho dovuto cercare sul forum di Qt e ho trovato un workaround , inizialmente ho provato ad aggiungere questo header: <arm_acle.h>, ma comunque dava errore, poi ho risolto con una soluzione di un utente che ha aggiunto le seguenti flag nel file. pro: QMAKE_CXXFLAGS += -Wno-error=implicit-function-declaration

QMAKE_CXXFLAGS += -Wno-implicit-function-declaration, ovviamente nel docker non vi era questo problema, stessa cosa per l'uso di signals ecc. che ho dovuto sostituire con Q_SIGNALS, come per slot ed emit.

Inoltre sempre passando da MacOS a docker, oltre agli elementi grafici differenti, che dipendono dal sistema operativo, mi uscivano delle parti di bottoni ecc. completamente nere, queste si sono risolte quando ho spostato gli elementi di stile da definizioni in-line nel codice con setStyleSheet al file QSS, qua onestamente non ho trovato la motivazione del problema.